Agentic Translation System

Design Report

Software Architecture and Design

GitHub Repository:

https://github.com/AMSONI777/Agentic-Translation-System

Created By:

Amit Soni

1. System Overview

The Agentic Translation System is a web-based application that enables users to translate text and audio across 16+ languages using an agentic workflow architecture. The system decomposes the translation problem into specialized agents coordinated by a central orchestrator, following SOLID design principles throughout.

The system supports two LLM backends interchangeably:

- CmsAI API - the instructor-provided on-campus server at http://cmsai:8000/generate/
- Groq API - a free commercial cloud API using LLaMA 3.3 70B for translation and Whisper Large v3 for audio transcription

Switching between backends requires changing a single line in the configuration file. All other code remains identical.

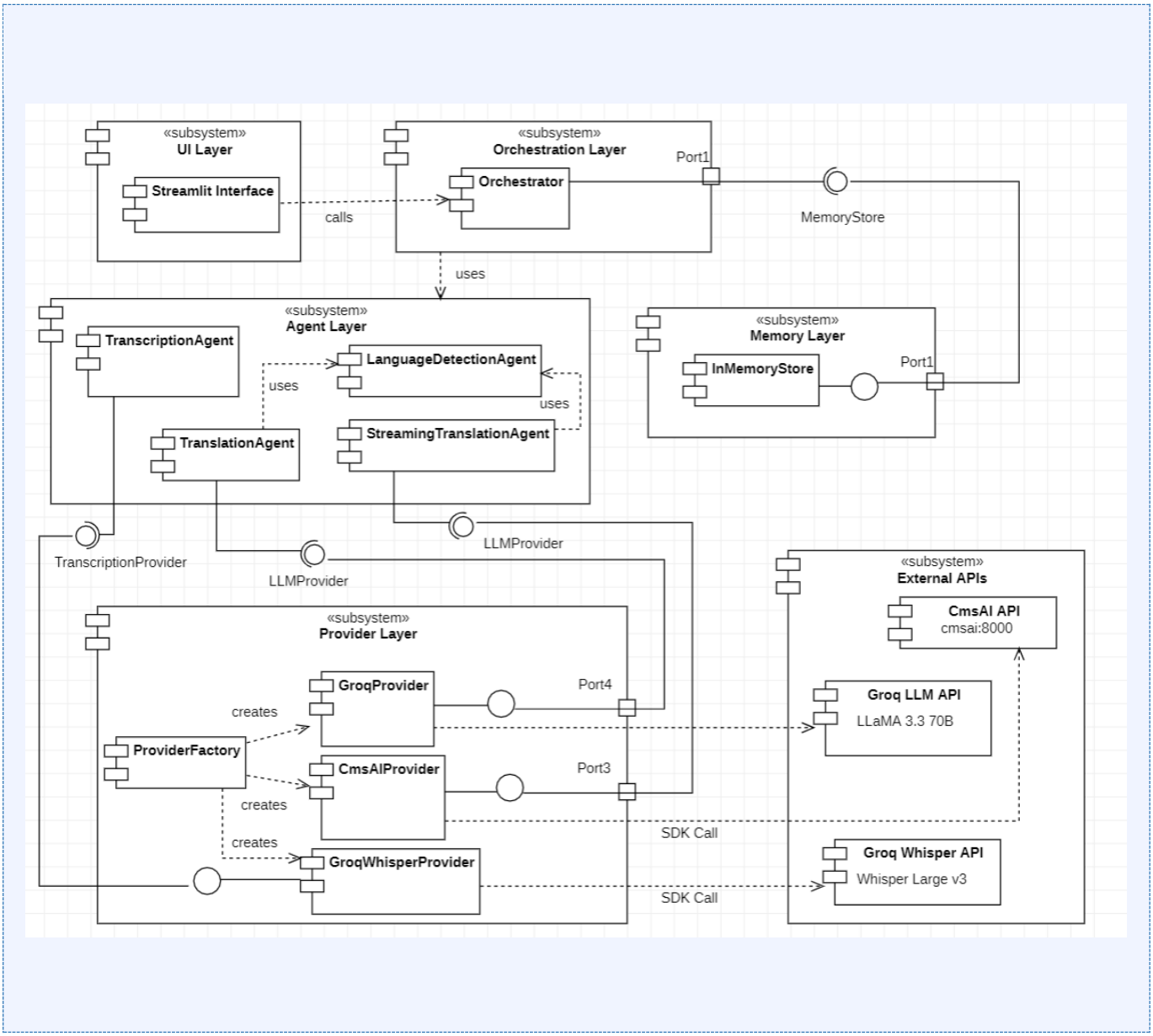
Feature	Status	Backend Used
Text Translation	Implemented	CmsAI or Groq LLM
Audio Translation	Implemented	Groq Whisper + CmsAI or Groq LLM
Real-Time Voice (Bonus)	Implemented	Groq Whisper + Groq Streaming
Conversation Memory (Bonus)	Implemented	In-Memory (session-based)

2. UML Diagrams

The following three UML diagrams describe the system structure and deployment. Each diagram is explained in detail below.

2.1 Component Diagram

The component diagram shows how the system is decomposed into collaborating modules and how the agentic workflow is coordinated across layers.



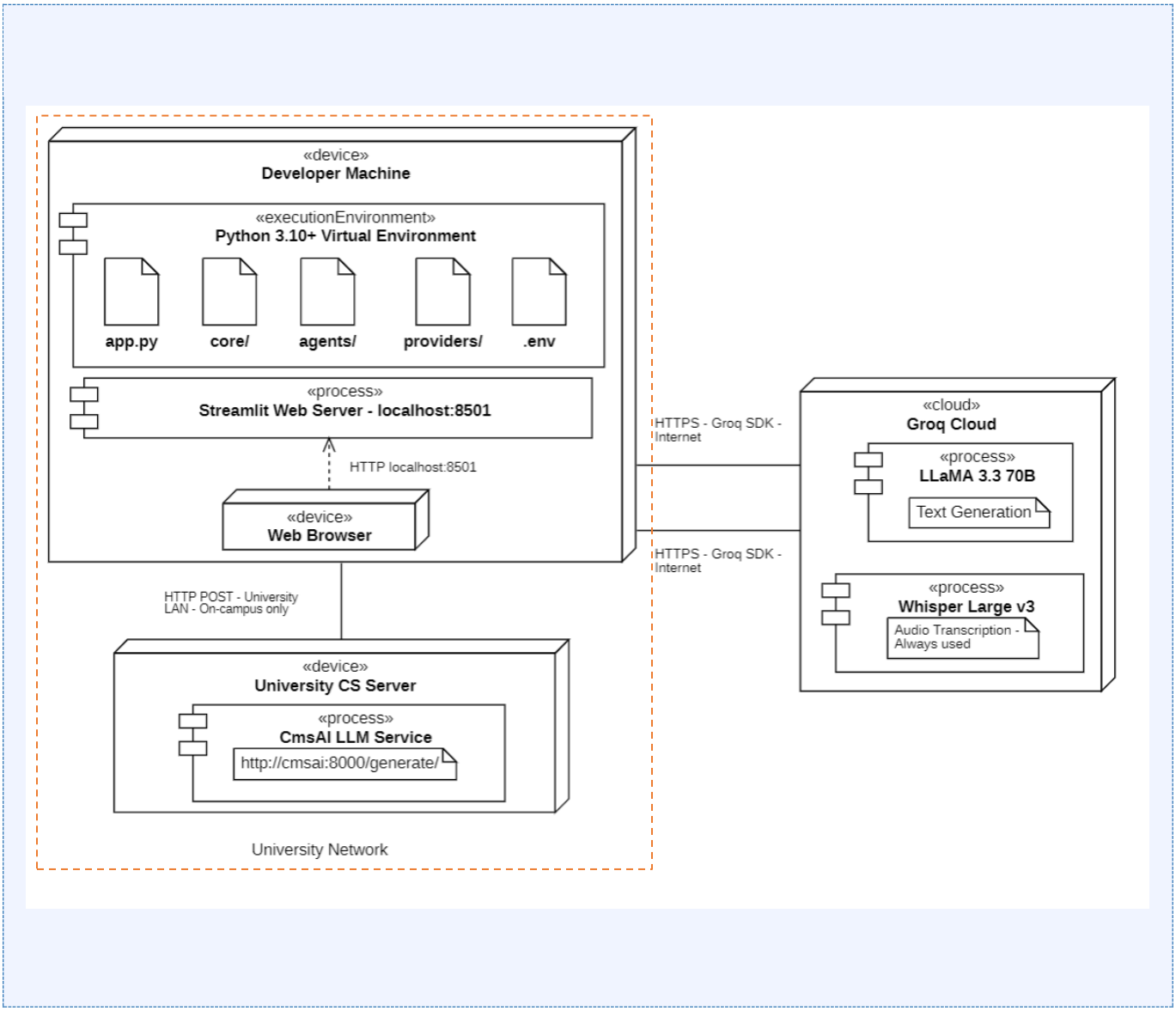
Key points shown in the diagram:

- UI Layer (app.py) sits at the top and communicates only with the Orchestrator, it never imports agents or providers directly. This enforces Dependency Inversion at the UI boundary.
- Orchestration Layer contains the Orchestrator which is the sole coordinator of all agents. It is the only component aware of all agents simultaneously.
- Agent Layer contains four specialized components - TranslationAgent, TranscriptionAgent, StreamingTranslationAgent, and LanguageDetectionAgent. Each agent has a single responsibility and no knowledge of the others.
- Provider Layer contains LLMProvider and TranscriptionProvider interfaces with their concrete implementations (CmsAIProvider, GroqProvider, GroqWhisperProvider) and the ProviderFactory which creates the correct concrete object from configuration.
- Memory Layer contains the MemoryStore interface and InMemoryStore implementation, used to persist conversation history per session.
- Data Models layer contains pure dataclasses that carry structured data between layers with no embedded logic.
- External APIs (CmsAI and Groq) sit outside the system boundary, reached only through the provider layer.

The agentic behavior is visible in the Agent Layer, each agent is independently testable, independently replaceable, and contributes to the workflow through the Orchestrator rather than calling each other directly.

2.2 Deployment Diagram

The deployment diagram shows how the system is distributed across hardware and execution environments.



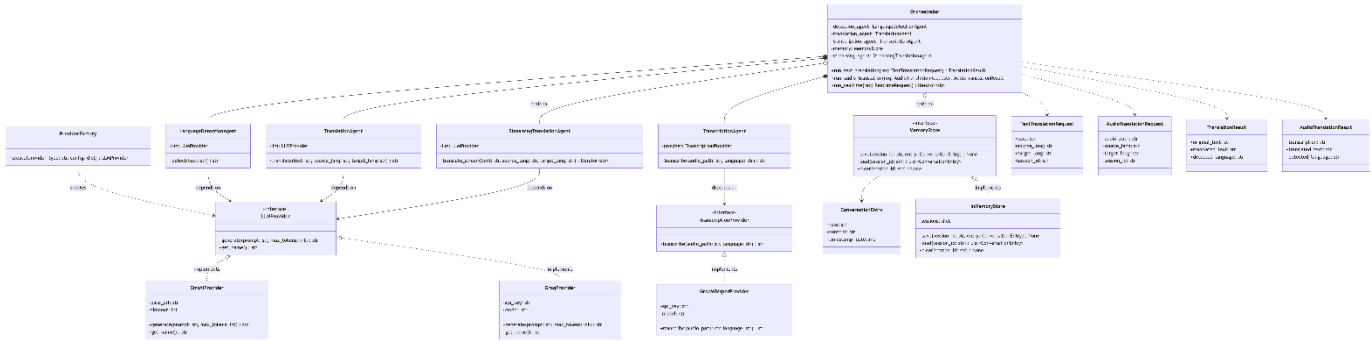
Three deployment nodes are shown:

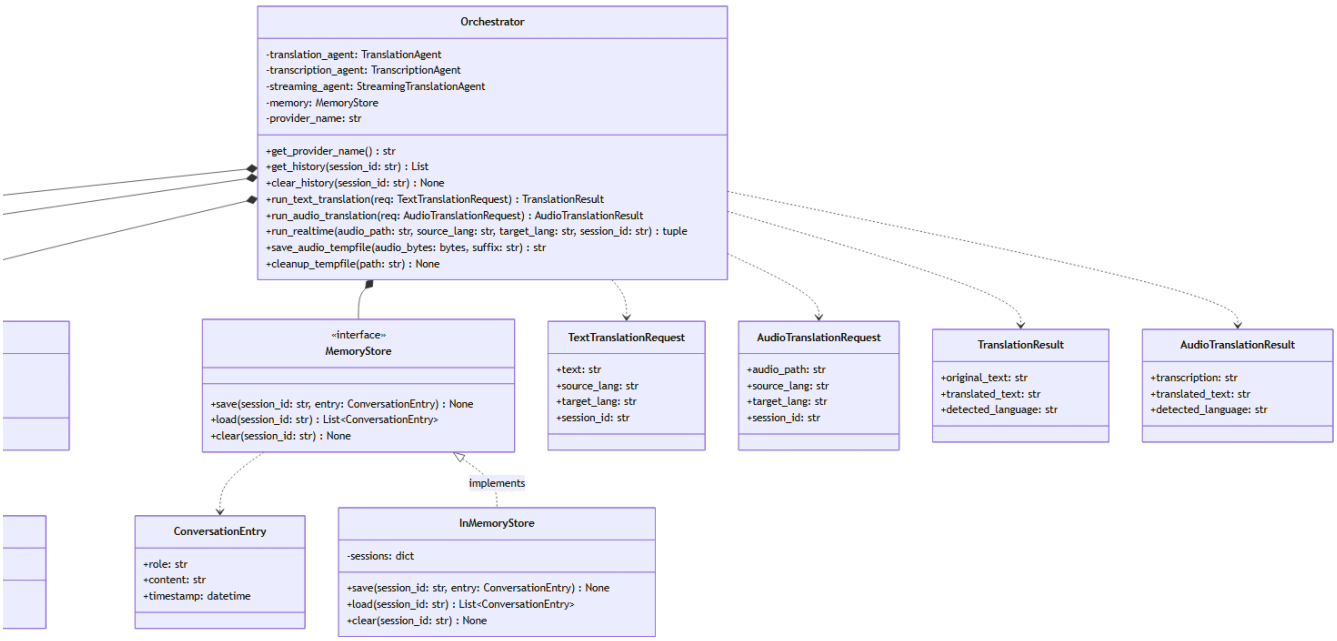
- Developer Machine (Windows PC) - hosts the Python virtual environment containing all application code. Streamlit runs a local web server on localhost:8501. The browser on the same machine accesses the UI through this port.
- University CS Server (On-Campus) - hosts the CmsAI LLM service provided by the instructor, accessible at http://cmsai:8000/generate/ over the university LAN. This node is only reachable when the developer machine is connected to the university network.
- Groq Cloud (Internet) - hosts two services accessed via HTTPS: LLaMA 3.3 70B for text generation and Whisper Large v3 for audio transcription. This node is reachable from anywhere with internet access.

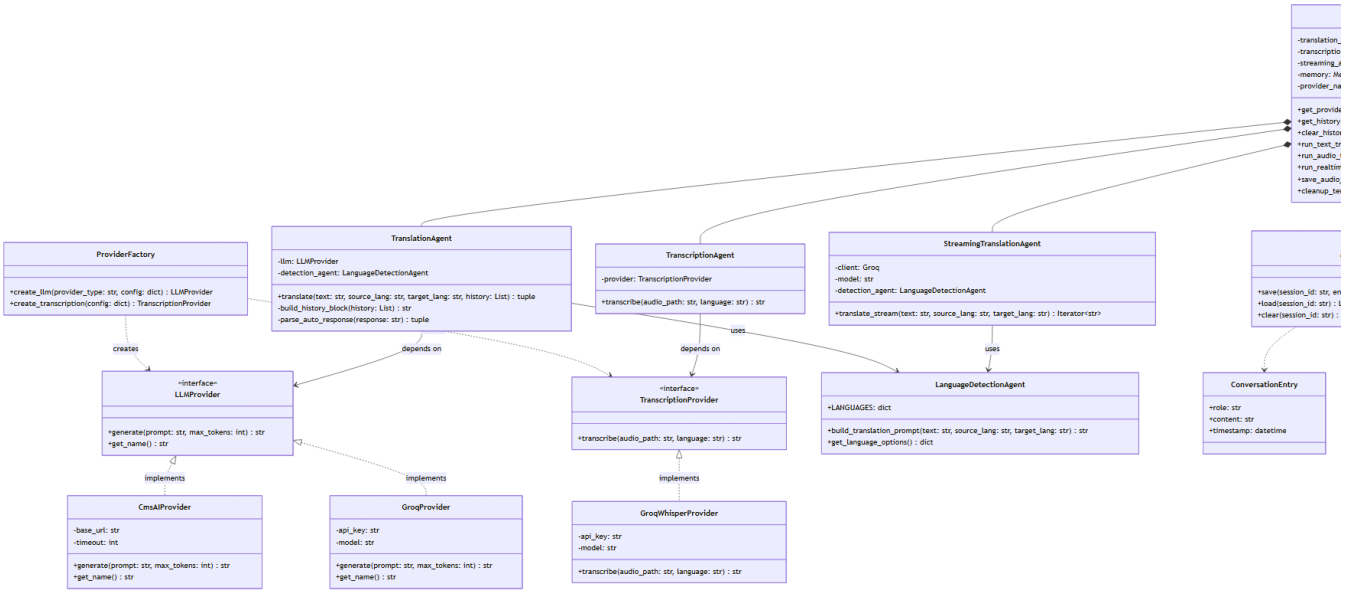
The university network boundary encloses both the developer machine and the CS server. Groq Cloud sits outside this boundary on the public internet. Audio transcription always routes to Groq Cloud regardless of which LLM backend is selected, because the CmsAI API is text-only.

2.3 Class Diagram

The class diagram shows all major classes, interfaces, relationships, attributes, and methods in the implementation.







Key relationships shown in the diagram:

- **Inheritance / Implementation:** CmsAIProvider and GroqProvider implement LLMProvider. GroqWhisperProvider implements TranscriptionProvider. InMemoryStore implements MemoryStore. These are shown with open-triangle arrows.
- **Dependency:** TranslationAgent and StreamingTranslationAgent depend on LLMProvider (interface). TranscriptionAgent depends on TranscriptionProvider. These are shown with dashed arrows.
- **Composition:** Orchestrator composes TranslationAgent, TranscriptionAgent, StreamingTranslationAgent, and MemoryStore, meaning these are created and owned by the Orchestrator.
- **Usage:** ProviderFactory creates LLMProvider and TranscriptionProvider objects. TranslationAgent and StreamingTranslationAgent use LanguageDetectionAgent for prompt construction.
- **Data flow:** Orchestrator uses all Request and Result dataclasses. MemoryStore uses ConversationEntry.

The LanguageDetectionAgent has no LLMProvider dependency, it is a pure prompt-building utility. This is visible in the class diagram where it has no dependency arrow to LLMProvider, distinguishing it from the other agents.

3. Agentic Architecture

The system is structured into five logical layers. Each layer depends only on abstractions defined in the layer below it, never on concrete implementations.

Layer	Responsibility
UI Layer (app.py)	Streamlit interface. Only calls Orchestrator methods. Never imports agents or providers directly.
Orchestration Layer	Orchestrator coordinates workflow, manages memory, and routes data between agents.
Agent Layer	Four specialized agents each performing exactly one job.
Provider Layer	Concrete API implementations hidden behind interfaces. Only ProviderFactory knows concrete types.
Data Models (models.py)	Pure dataclasses with no logic. Passed between layers as structured data.

3.1 Agents

- LanguageDetectionAgent - No LLM dependency. Constructs the correct prompt based on source language selection. When Auto-Detect is chosen, instructs the LLM to detect and translate in a single call.
- TranslationAgent - Depends on LLMProvider interface. Builds prompt via LanguageDetectionAgent, makes one API call, parses the two-line response when auto-detect is used, and prepends conversation history when memory is active.
- TranscriptionAgent - Depends on TranscriptionProvider interface. Converts audio file path to transcription string by delegating to GroqWhisperProvider.
- StreamingTranslationAgent - Used for real-time bonus. Calls Groq API with stream=True, yielding tokens one by one as a Python generator for word-by-word display.

3.2 Orchestrator

The Orchestrator defines three workflow methods:

- run_text_translation() - loads history, calls TranslationAgent, saves result to memory
- run_audio_translation() - calls TranscriptionAgent then TranslationAgent, saves both to memory
- run_realtime() - calls TranscriptionAgent then StreamingTranslationAgent, stores full translation after stream ends

4. Workflow Description

4.1 Text Translation

The text translation workflow begins when the user types text into the input area, selects a source language (or chooses Auto-Detect), and selects a target language. When the Translate button is clicked, the UI constructs a TextTranslationRequest dataclass containing the text, language selections, and the current session ID, then passes it to the Orchestrator.

The Orchestrator first loads any existing conversation history for that session from the MemoryStore. This history is passed to the TranslationAgent so that past exchanges can provide context to the LLM. The LanguageDetectionAgent then constructs the appropriate prompt, if Auto-Detect was selected, the prompt instructs the LLM to identify the source language and return it alongside the translation on separate lines. If a specific source language was chosen, the prompt simply requests a direct translation with no detection step.

The TranslationAgent sends this prompt to the active LLM backend in a single API call. The response is parsed, if auto-detection was used, the detected language label and the translation text are extracted separately. The Orchestrator then saves both the original input and the translated result to the MemoryStore under the current session. Finally, a TranslationResult is returned to the UI, which displays the translated text and the detected language label if applicable.

4.2 Audio Translation

The audio translation workflow begins when the user uploads an audio file and selects language preferences. Before any API call is made, the Orchestrator writes the uploaded audio bytes to a temporary file on disk. This is necessary because the Whisper transcription API requires a real file path, it cannot accept raw in-memory bytes directly.

The Orchestrator then calls the TranscriptionAgent, which forwards the temporary file path to the GroqWhisperProvider. Groq's hosted Whisper Large v3 model processes the audio and returns the full transcription as a text string. This is a blocking call; the system waits for the complete transcription before proceeding.

Once the transcription is available, it is passed to the TranslationAgent, which follows the exact same prompt-construction and LLM-call process as the text translation workflow. The Orchestrator saves both the transcription and the translated result to the MemoryStore for the current session.

An AudioTranslationResult is returned to the UI containing both the transcription and the translation. These are displayed side by side so the user can see the intermediate speech-to-text output alongside the final translated text. After the result is displayed, the temporary audio file is deleted from disk in a finally block, ensuring cleanup even if an error occurred during processing.

4.3 Real-Time Voice Translation (Bonus)

The real-time workflow begins when the user presses the microphone icon in the Real-Time tab. Streamlit's built-in audio input widget captures the voice recording and returns the audio as bytes once the user stops recording. These bytes are written to a temporary WAV file on disk, the same way as the audio upload workflow.

The Orchestrator calls the TranscriptionAgent, which sends the file to Groq Whisper and waits for the full transcription to return. This is still a blocking call, real-time behaviour in this implementation refers to the translation output being streamed word by word, not the transcription itself.

Once the transcription is available, the Orchestrator passes it to the StreamingTranslationAgent. Unlike the TranslationAgent, the streaming agent calls the Groq LLM API with streaming enabled, which causes the model to return its response as a continuous stream of small text chunks called tokens, rather than waiting for the full response to be ready. The StreamingTranslationAgent yields each token one at a time through a Python generator.

A wrapper generator inside the Orchestrator consumes this stream, forwarding each token to the UI while simultaneously accumulating the complete translation string in memory. The Streamlit UI updates a placeholder element with each incoming token, producing the word-by-word streaming effect visible to the user. Once the stream is fully exhausted and the complete translation is assembled, the full exchange, both the transcription and the translation, is saved to the MemoryStore under the current session ID.

The UI renders the output in two clearly separated blocks: the detected language and transcription first, followed by the streamed translation. The temporary audio file is deleted after the stream completes.

5. Model and API Strategy

5.1 LLM Backend Isolation

The LLMProvider interface in providers/base.py defines the contract for all LLM backends with only two methods: generate() and get_name(). No agent, orchestrator, or UI component imports a concrete provider class. ProviderFactory is the only place that knows which concrete class to instantiate, based on the PROVIDER value in .env.

Backend	Class	When Used
CmsAI API	CmsAIProvider	PROVIDER=cmsai (on-campus only)
Groq LLM	GroqProvider	PROVIDER=groq (default, anywhere)
Groq Whisper	GroqWhisperProvider	Always - audio transcription only

5.2 Switching Backends

Open the .env file and change PROVIDER:

```
PROVIDER=groq      # off-campus (default)
PROVIDER=cmsai     # on-campus professor API
```

The Streamlit sidebar also provides a dropdown to switch at runtime without restarting the app. Audio transcription always uses Groq Whisper regardless of the selected LLM backend, because the CmsAI API is text-only with no audio endpoint.

6. SOLID Design Principles

Principle	Applied As
Single Responsibility	Every class has one job. TranslationAgent only translates. TranscriptionAgent only transcribes. LanguageDetectionAgent only builds prompts. Orchestrator only coordinates. Provider classes only make API calls.
Open / Closed	Adding a new LLM backend requires one new class implementing LLMProvider and one line in ProviderFactory. No existing code changes. Same applies for transcription providers.
Liskov Substitution	CmsAIProvider and GroqProvider are fully interchangeable wherever LLMProvider is expected. TranslationAgent works identically regardless of which is injected.
Interface Segregation	Three separate interfaces: LLMProvider, TranscriptionProvider, and MemoryStore. No class implements methods unrelated to its function.
Dependency Inversion	app.py calls only Orchestrator. Orchestrator calls only agents. Agents call only provider interfaces. Concrete types are known only to ProviderFactory.

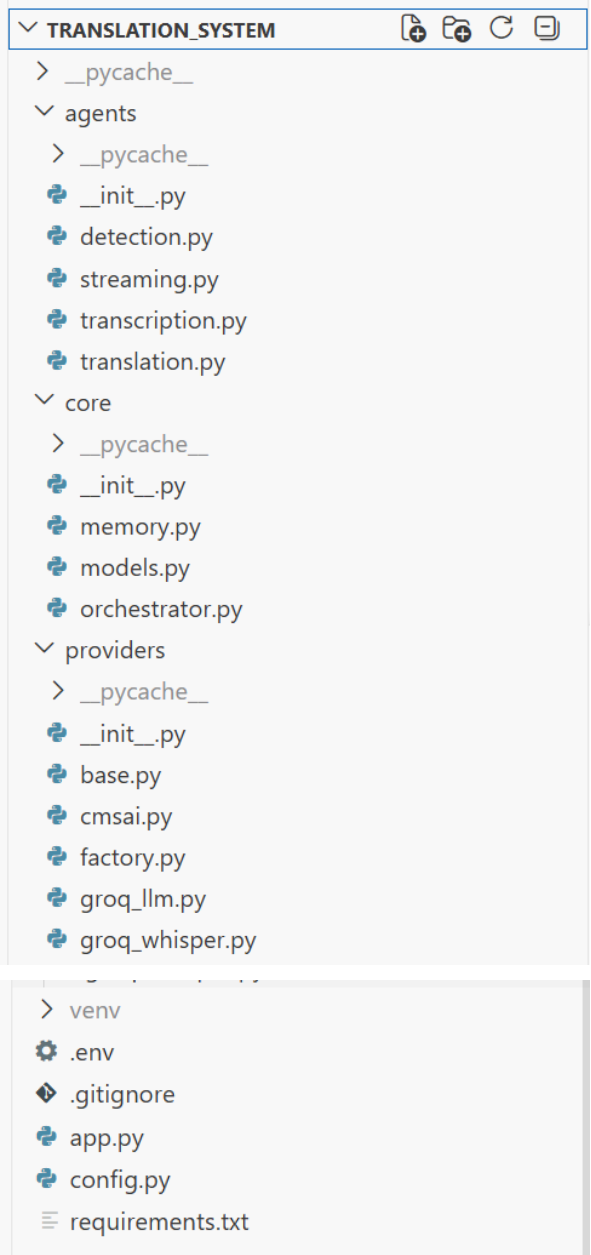
7. Error Handling and Robustness

A custom exception ProviderUnavailableError is defined in providers/cmsai.py and reused across all provider classes. This ensures a single exception type represents all backend failures. The system does not auto-fallback between providers, errors surface as a red Streamlit banner and the user switches manually.

Error Condition	Handling
CmsAI unreachable (off-campus)	ConnectionError caught, raises ProviderUnavailableError: Are you on campus? Try switching to Groq.
CmsAI timeout	Timeout caught, raises ProviderUnavailableError with clear message.
Non-200 HTTP status	Status code included in ProviderUnavailableError message.
Unexpected JSON format	KeyError caught, raises ProviderUnavailableError showing actual response.
Groq SDK failure	All exceptions caught, wrapped in ProviderUnavailableError with Groq error detail.
Audio file missing	FileNotFoundError caught in GroqWhisperProvider with descriptive message.
Empty user input	Caught in UI before any API call. Shows Streamlit warning.
Same source and target	Caught in UI before any API call. Shows Streamlit warning.
LLM ignores response format	Auto-detect parsing degrades gracefully: full response returned as translation, detected language set to unknown. No crash.
Temp file cleanup	Always runs in a finally block. Temp audio file deleted even when errors occur.

8. Project Structure

In the VSCode IDE,



providers/base.py - Two abstract interfaces: LLMProvider and TranscriptionProvider. No implementation, just contracts that all provider classes must follow.

providers/cmsai.py - Connects to the professor's on-campus API at cmsai:8000. Also defines ProviderUnavailableError which is reused across all providers.

providers/groq_llm.py - Connects to Groq's cloud API using LLaMA 3.3 70B for text generation.

providers/groq_whisper.py - Connects to Groq's Whisper Large v3 model for converting audio files to text.

providers/factory.py - Reads the provider type from config and creates the correct LLM and transcription provider objects. Only place in the codebase that knows about concrete provider classes.

agents/detection.py - Builds the translation prompt. No API calls. Handles the difference between auto-detect and explicit source language selection.

agents/translation.py - Makes the single LLM API call for translation. Parses the detected language and translation from the response. Prepends conversation history when memory is active.

agents/transcription.py - Wraps the transcription provider. Converts an audio file path into a transcribed text string.

agents/streaming.py - Handles real-time streaming translation. Yields tokens one by one from the Groq streaming API for word-by-word display.

core/models.py - All dataclasses used across the system: request objects, result objects, and ConversationEntry for memory.

core/memory.py - MemoryStore abstract interface and InMemoryStore implementation for storing conversation history per session.

core/orchestrator.py - Coordinates all agents and memory. Defines the three main workflows: text translation, audio translation, and real-time. Also handles temp file creation and cleanup.

config.py - Reads the .env file and exposes all configuration values as a single Config class.

app.py - The entire Streamlit UI. Four tabs: Text Translation, Audio Translation, History, Real-Time. Only file that imports Streamlit. Only calls the Orchestrator — never agents or providers directly.

.env - Two lines: PROVIDER (groq or cmsai) and GROQ_API_KEY.

requirements.txt - Four dependencies: streamlit, groq, requests, python-dotenv.

9. Setup and Testing Guide for Graders

The .env file is included in the repository and contains a working Groq API key. No account creation or API key setup is required. Simply extract, install, and run.

Step 1 - Prerequisites

- Python 3.10 or higher - verify with: python --version
- Git - verify with: git --version
- pip - included with Python

Step 2 - Extract and Open the Project

Extract the submitted ZIP file to any location on your machine.

Then open your terminal and navigate into the extracted folder:

Windows (PowerShell):
```
cd path\to\Agentic-Translation-System
```

Mac / Linux:
```
cd path/to/Agentic-Translation-System
```

You should see files like app.py, requirements.txt, and, .env inside the folder before proceeding.

Step 3 - Create Virtual Environment

Windows (PowerShell):

```
python -m venv venv
venv\Scripts\activate
```

Mac / Linux:

```
python -m venv venv
source venv/bin/activate
```

You should see (venv) appear in your terminal prompt after activation.

Step 4 - Install Dependencies

```
pip install -r requirements.txt
```

Step 5 - Run the Application

```
streamlit run app.py
```

The application will open automatically in your browser at http://localhost:8501

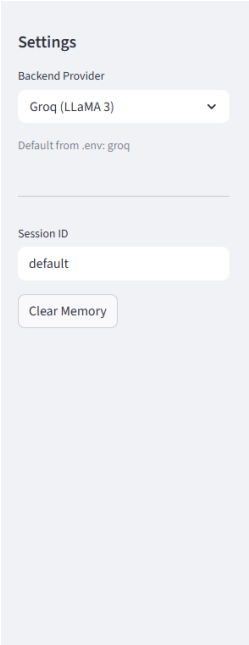
Step 6 - Testing Core Requirements

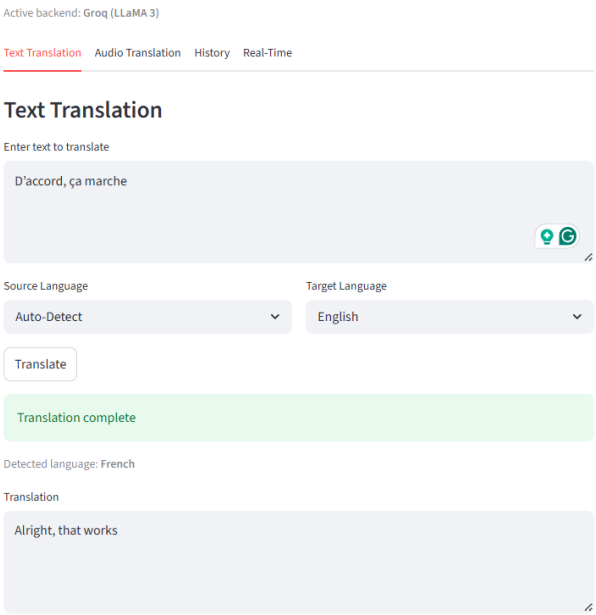
Test 1: Text Translation

- Click the Text Translation tab
- Type any text (example: Bonjour, comment allez-vous?)
- Set Source Language to Auto-Detect, Target Language to English
- Click Translate
- **Expected:** Green success banner, detected language label, and translated text

Screenshots:

Using Groq:



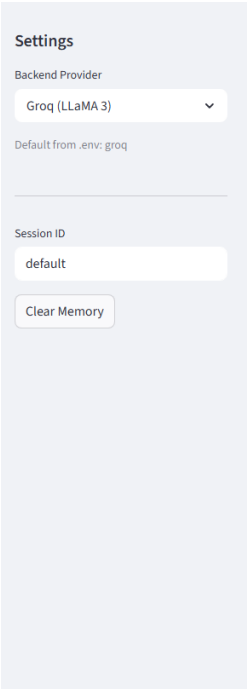


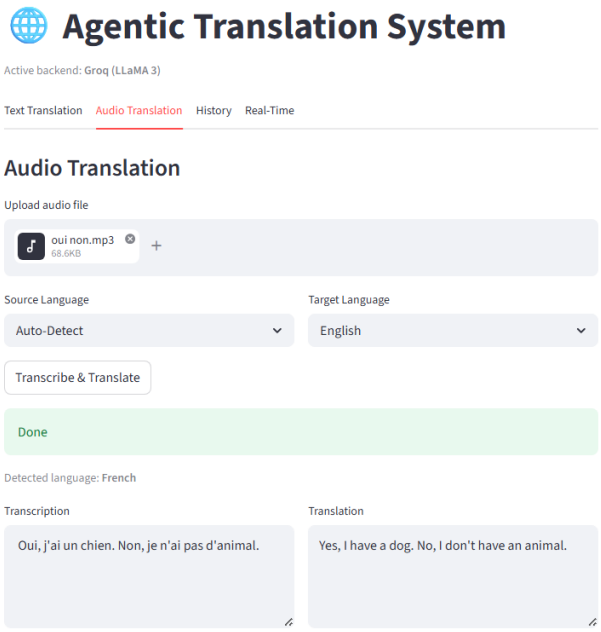
Test 2: Audio Translation

- Click the Audio Translation tab
- Upload any audio file (mp3, wav, m4a, or webm)
- Set Source Language to Auto-Detect, Target Language to English
- Click Transcribe & Translate
- **Expected:** Transcription box and translation box displayed side by side

Screenshots:

Using Groq:



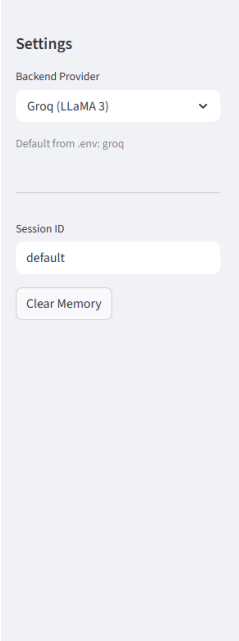


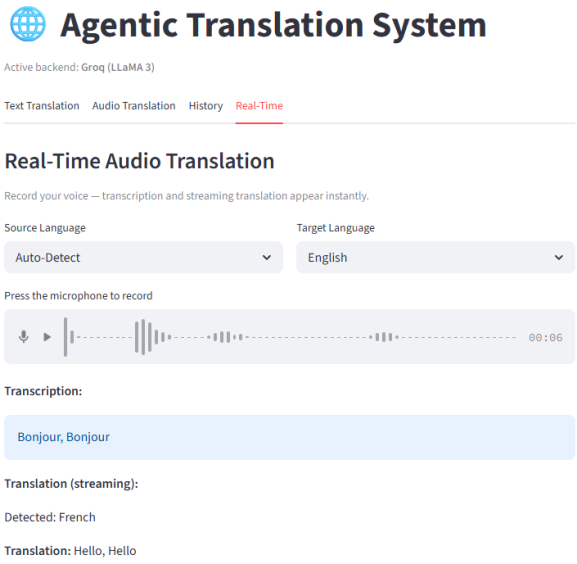
Test 3: Real-Time Voice (Bonus)

- Click the Real-Time tab
- Click the microphone icon, speak a sentence in any language, click stop
- **Expected:** Transcription appears in blue box, translation streams word by word below

Screenshots:

Using Groq:



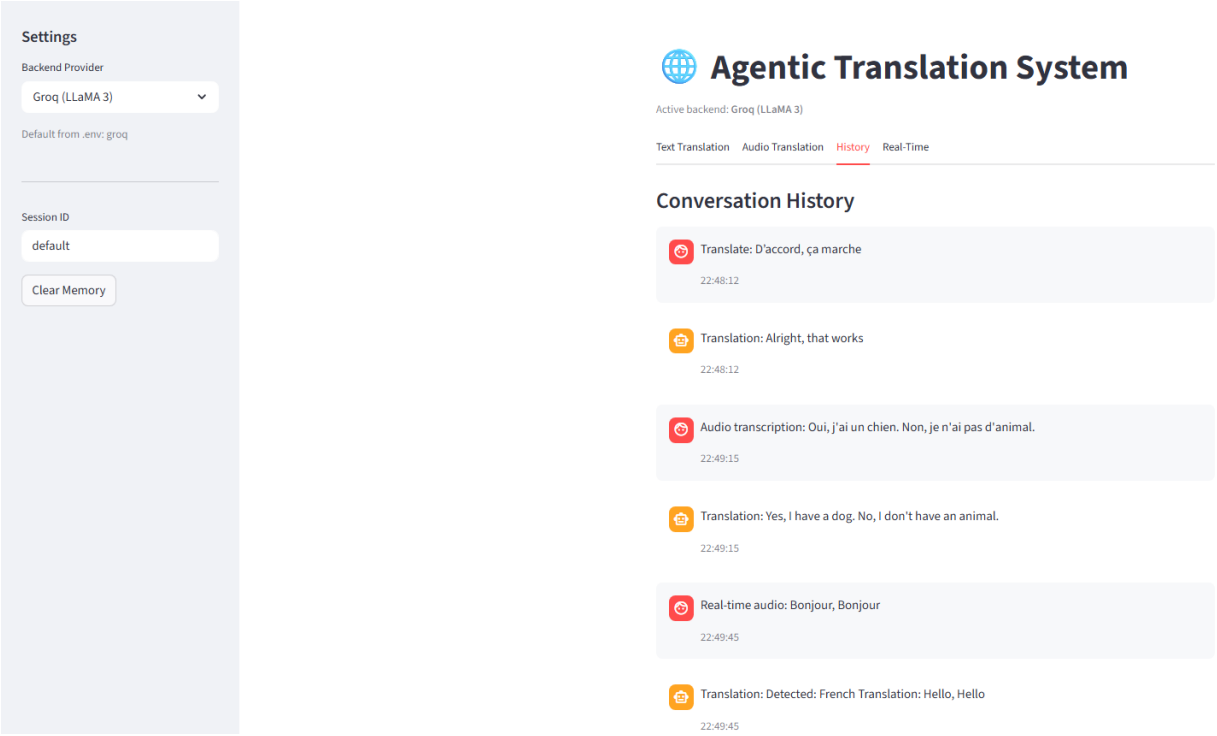


Test 4: Conversation History (Bonus)

- Perform two or three text translations
- Click the History tab
- Expected: All past translations shown as chat-style history with timestamps
- Click Clear Memory in the sidebar to reset

Screenshots:

Using Groq:

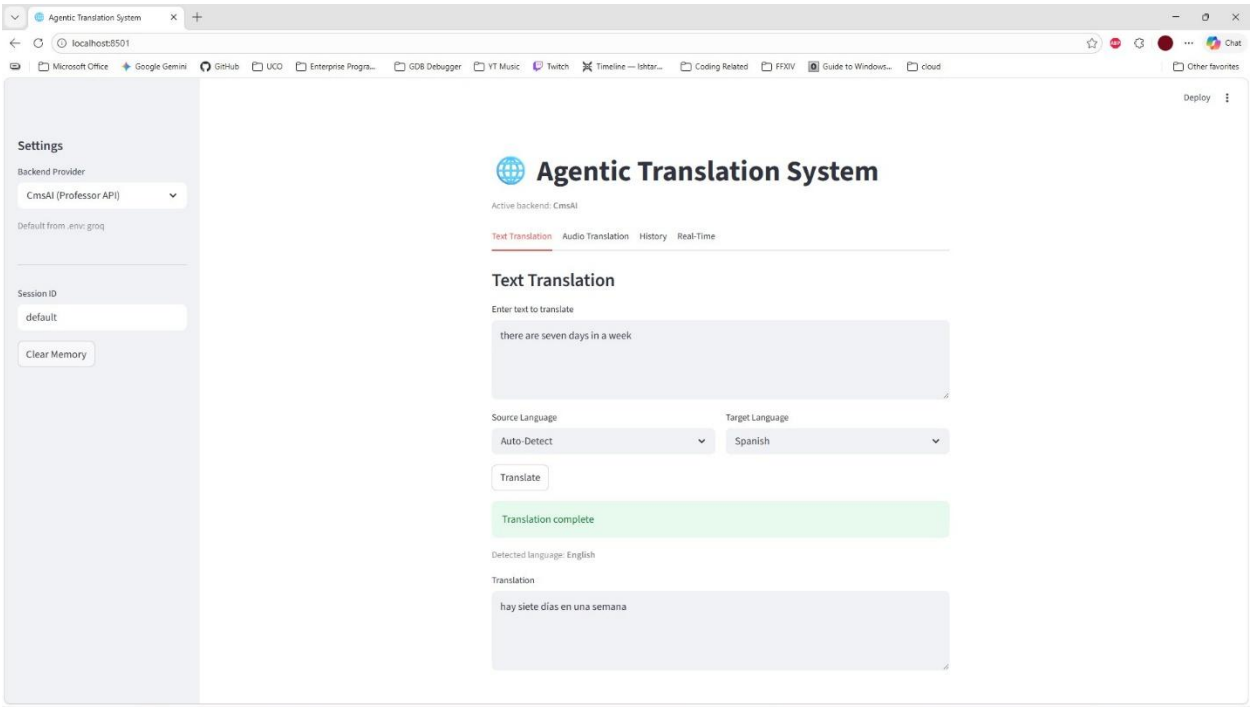


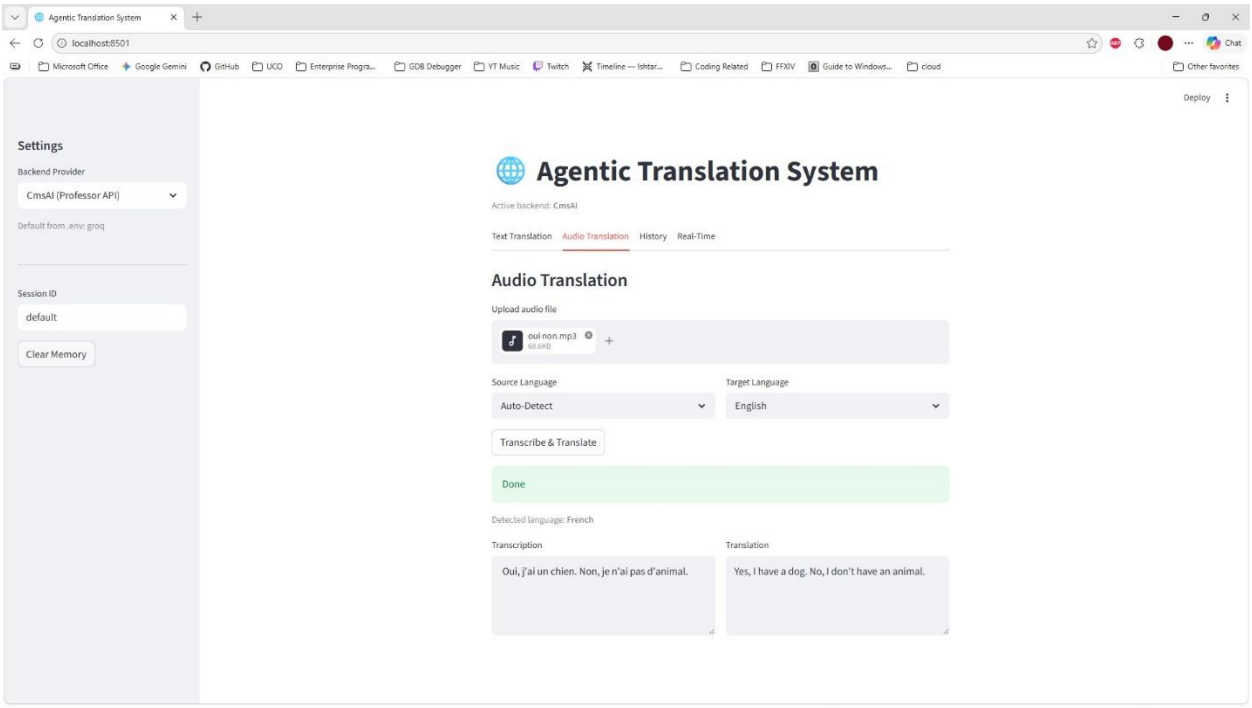
Step 7 - Testing the Professor API (On-Campus Only)

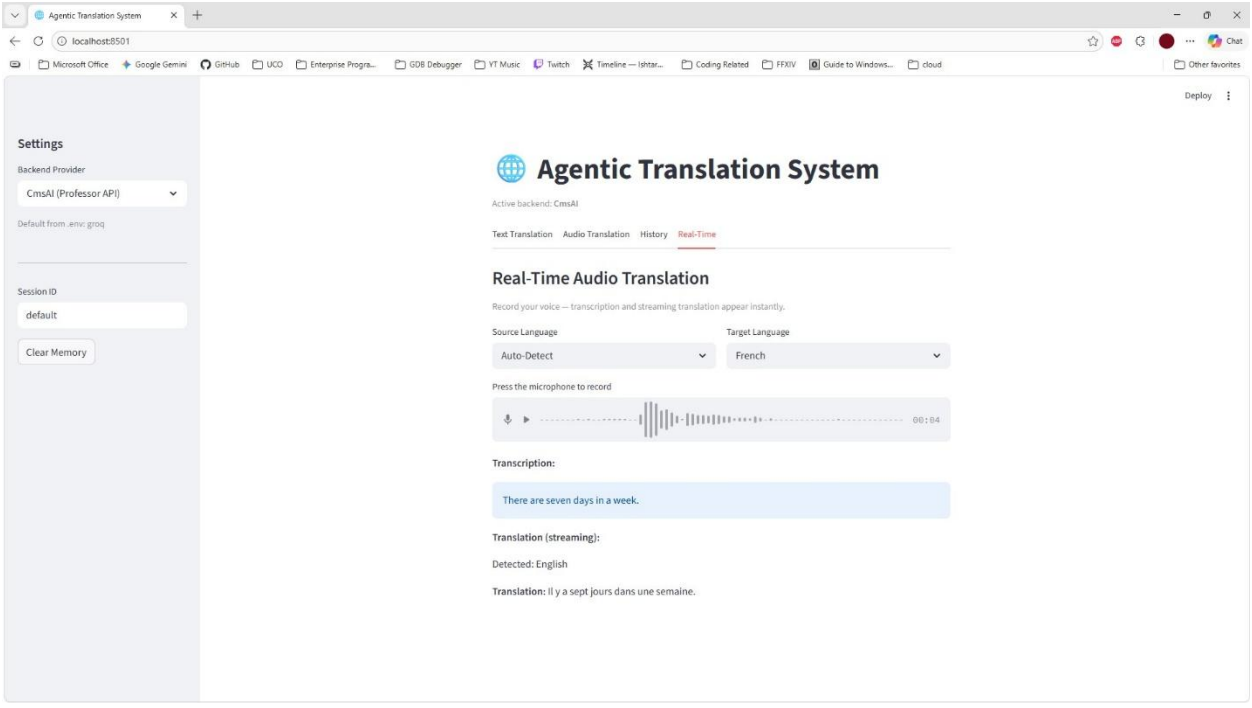
- Open the .env file in the project root
- Change PROVIDER=groq to PROVIDER=cmsai
- Save the file and restart: streamlit run app.py
- **In the sidebar,** confirm Backend Provider shows CmsAI (Professor API)
- Perform a text translation - if on campus and server is running, it will work normally
- If unreachable, a red error banner appears - switch back to Groq in the sidebar

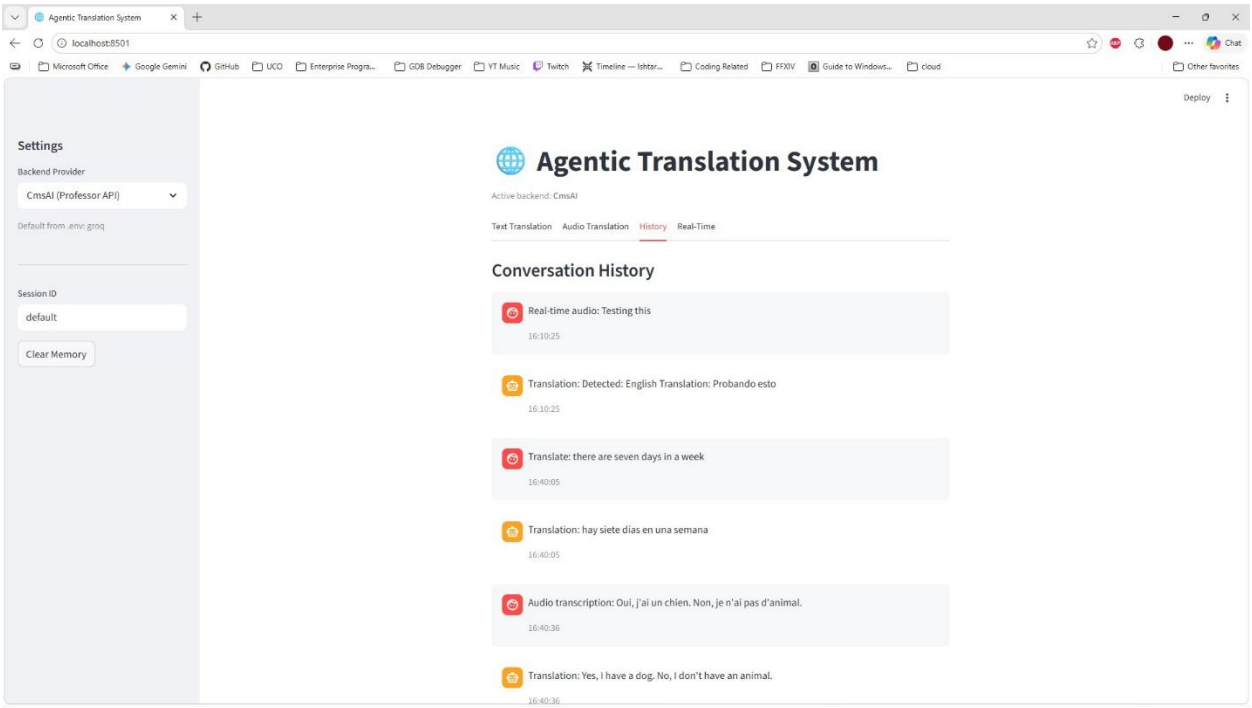
Screenshots:

Using CmsAI:




Agentic Translation System
localhost:8501
Settings
Backend Provider
CmsAI (Professor API)
Default from .env: groq
Session ID
default
Clear Memory
Deploy
Agentic Translation System
Active backend: CmsAI
Text Translation
Audio Translation
History
Real-Time
Audio Translation
Upload audio file
oui non.mp3
Source Language
Auto-Detect
Target Language
English
Transcribe & Translate
Done
Detected language: French
Transcription
Oui, j'ai un chien. Non, je n'ai pas d'animal.
Translation
Yes, I have a dog. No, I don't have an animal.


Agentic Translation System
localhost:8501
Settings
Backend Provider
CmsAI (Professor API)
Default from .env: groq
Session ID
default
Clear Memory
Deploy
Agentic Translation System
Active backend: CmsAI
Text Translation
Audio Translation
History
Real-Time
Real-Time Audio Translation
Record your voice — transcription and streaming translation appear instantly.
Source Language
Auto-Detect
Target Language
French
Press the microphone to record
Transcription:
There are seven days in a week.
Translation (streaming):
Detected: English
Translation: Il y a sept jours dans une semaine.



Default Configuration

```
# .env file (already configured - no changes needed for Groq)
PROVIDER=groq
GROQ_API_KEY=<included in repository>
```

Dependencies Reference

Package	Version	Purpose
streamlit	Latest	Web UI framework
groq	Latest	Groq SDK for LLM and Whisper API
requests	Latest	HTTP calls to CmsAI professor API
python-dotenv	Latest	Reads .env configuration file

10. Extensibility

The **SOLID** design ensures all future features can be added without modifying existing code:

Feature to Add	What to Create	Existing Code Changed
New LLM backend (e.g. OpenAI)	One new class implementing LLMProvider + one line in ProviderFactory	None
New transcription service	One new class implementing TranscriptionProvider + one line in factory	None
Persistent memory (database)	New class implementing MemoryStore interface	None
Text-to-speech output	New TTSProvider interface + TTSAgent + UI button	None
New UI tab	Add tab in app.py only	None